Effiziente Sortieralgorithmen

Einführung in die Programmierung 1

Sommersemester 25

TU WIEN

Überblick

- Mergesort
- Quicksort
- Sortieren in linearer Zeit
- Ausblick

Mergesort

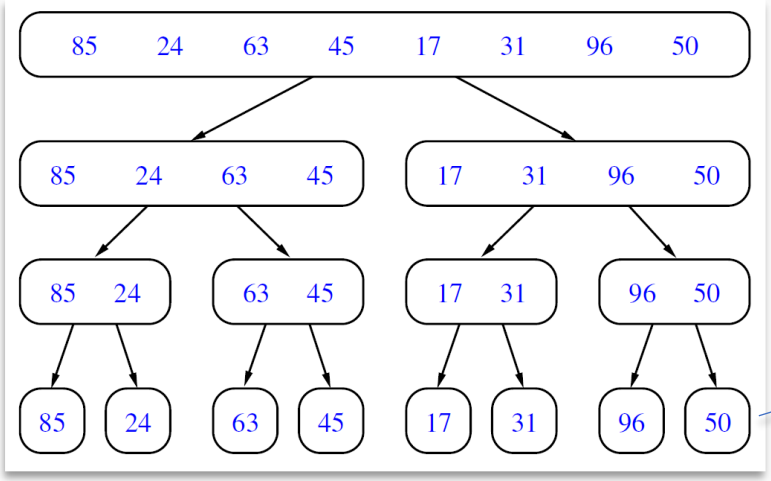
Teile-und-herrsche-Verfahren

- Allgemeines Prinzip für Algorithmenentwurf
  - Teile ein Problem in Teilprobleme auf
  - Löse jedes Teilproblem unabhängig für sich
  - Kombiniere die Teillösungen zu einer Gesamtlösung für das ursprüngliche Problem
- Im Englischen unter *Divide-and-Conquer* bekannt
- Wenn ein Teile-und-herrsche-Verfahren für ein Problem bekannt ist
  - Lässt es sich rekursiv implementieren
  - Basisfall ist erreicht, wenn das Teilproblem so einfach ist, dass die Lösung sofort angegeben werden kann

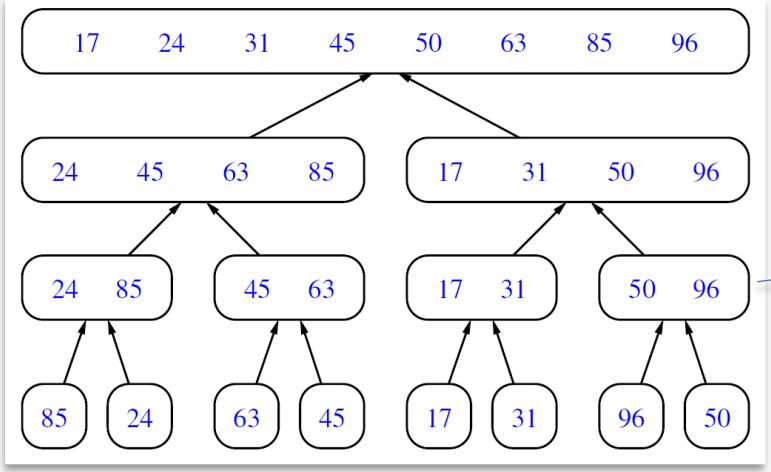
Mergesort – Idee

- Ein Teile-und-herrsche-Verfahren
- Vorgehen
  - Teile ursprüngliche Menge an Daten in zwei Hälften
  - Sortiere jede Hälfte für sich (rekursiv)
  - Verschmelze die beiden sortierten Hälften
    - Vergleiche immer wieder die aktuell vordersten Elemente der beiden sortierten Hälften
    - Entnimm (bzw. verarbeite) das kleinere der beiden
    - Am Ende ergibt sich so eine geordnete Sequenz aus den zwei geordneten Hälften
- Wird auch als Top-Down-Mergesort bezeichnet

Visuelles Beispiel für Ablauf (Überblick)

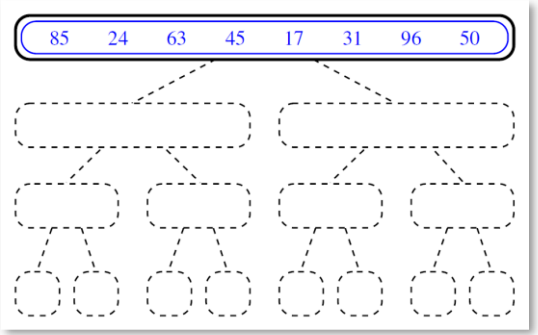


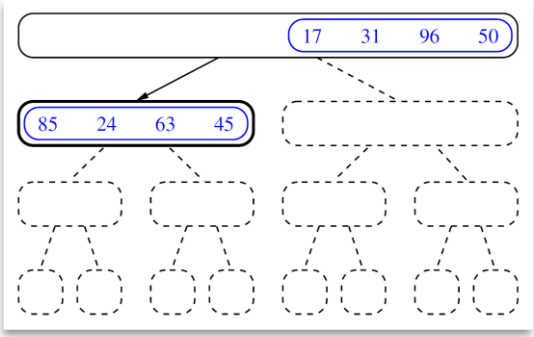
Solange aufteilen, bis nur mehr ein Element betrachtet wird

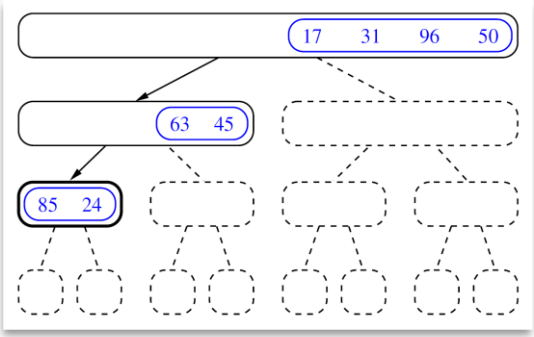


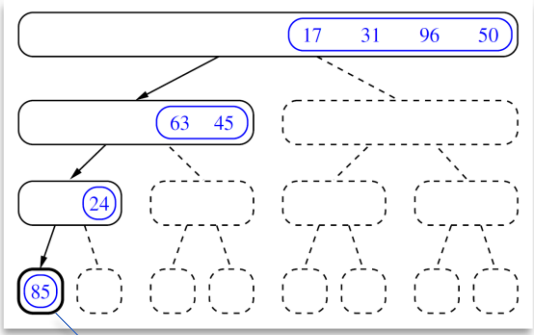
Verschmelzen von jeweils zwei sortierten Bereichen zu einem neuen sortierten Bereich

Beispiel – Rekursive Aufrufe (1)

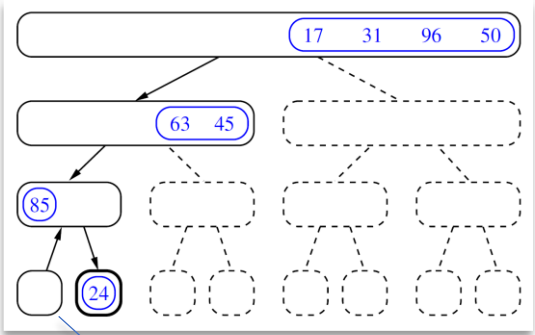




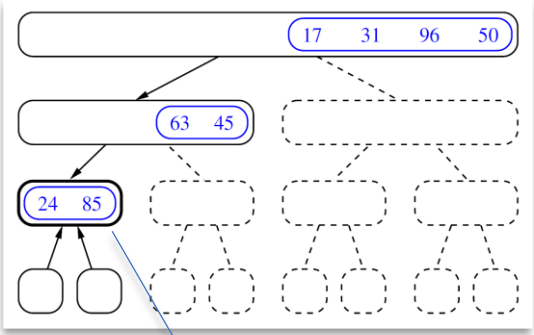




Aktueller Aufruf

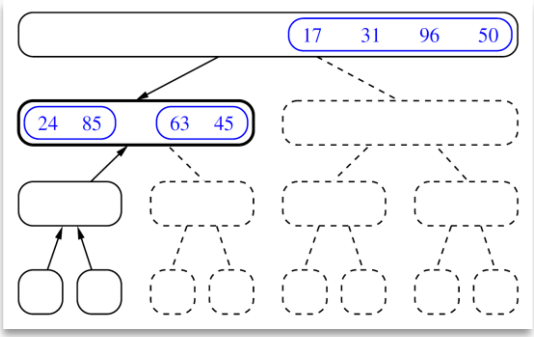


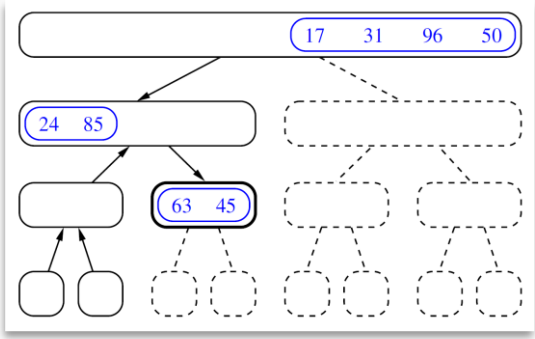
Beendeter Aufruf

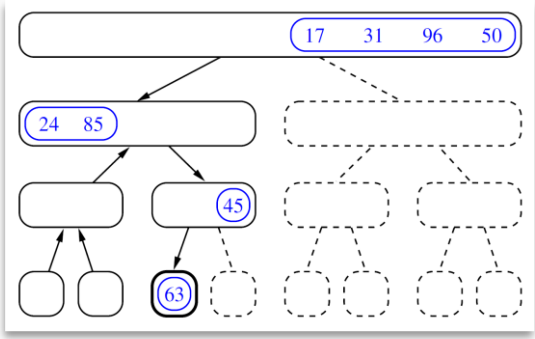


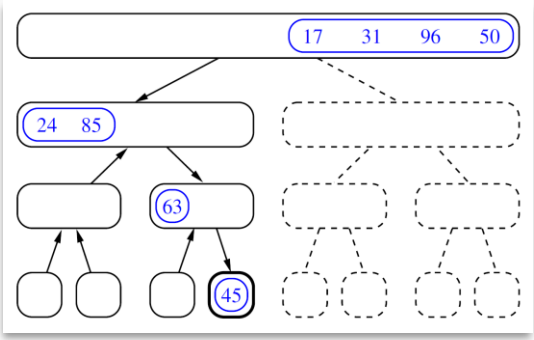
Nach Verschmelzen

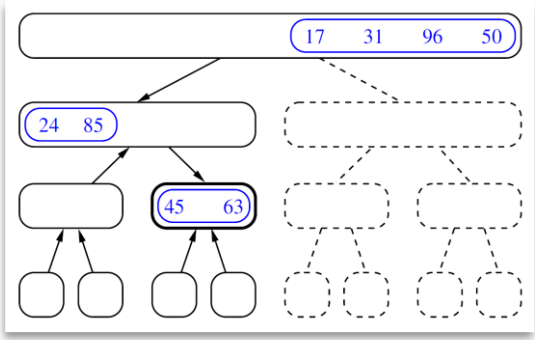
Beispiel – Rekursive Aufrufe (2)

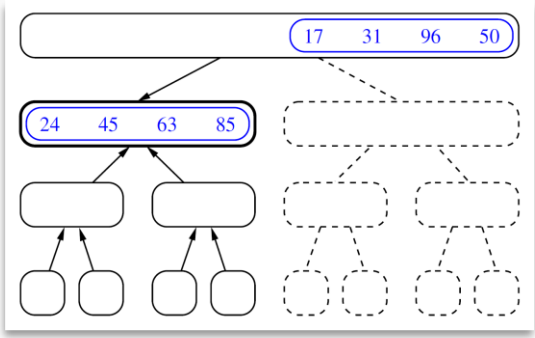




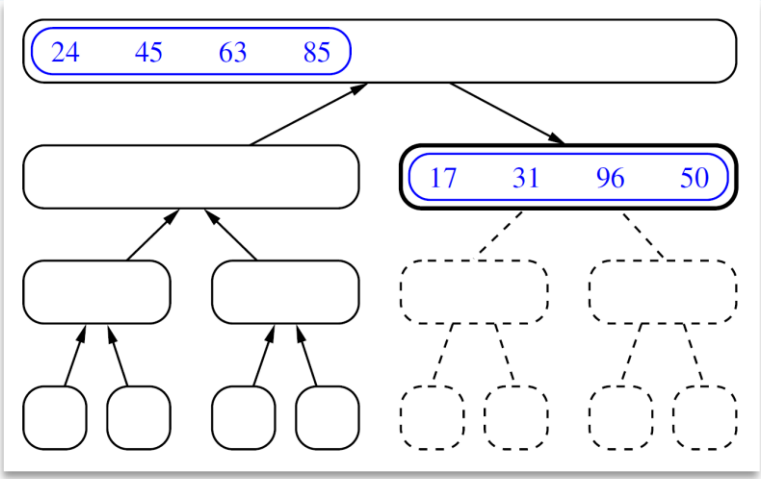


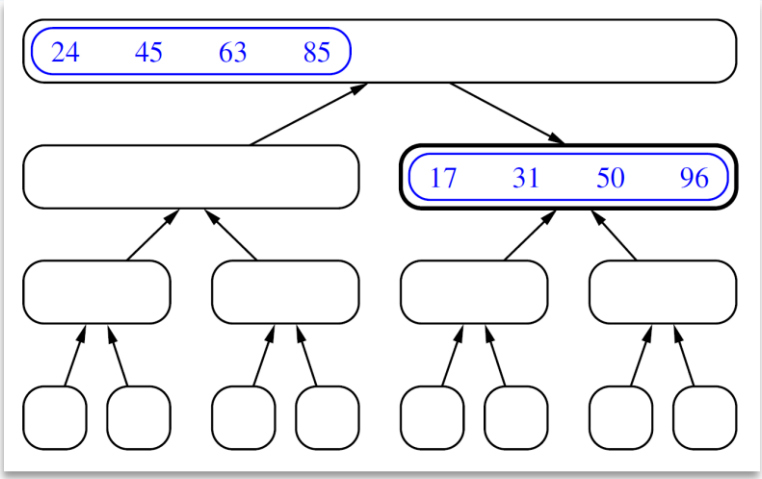


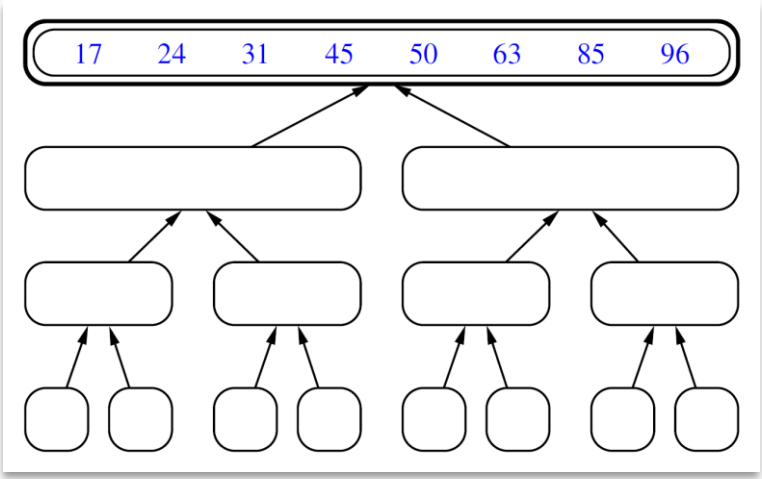




Beispiel – Rekursive Aufrufe (3)







Mergesort – Implementierung

```
private static void mergeSort(int[] data) {
    int[] help = new int[data.length];
    mergeSort(data, help, 0, data.length - 1);
}

private static void mergeSort(int[] data, int[] help, int lo, int hi) {
    if (hi <= lo) return;
    int mid = lo + (hi - lo) / 2;
    mergeSort(data, help, lo, mid);
    mergeSort(data, help, mid + 1, hi);
    merge(data, help, lo, mid, hi);
}

private static void merge(int[] data, int[] help, int lo, int mid, int hi) {
    for (int k = lo; k <= hi; k++) {
        help[k] = data[k];
    }
    int i = lo, j = mid + 1;
    for (int k = lo; k <= hi; k++) {
        if (i > mid) data[k] = help[j++];
        else if (j > hi) data[k] = help[i++];
        else if (help[j] < help[i]) data[k] = help[j++];
        else data[k] = help[i++];
    }
}
```

Aufruf mit einem Parameter (wie bisher)

Mergesort
Zuerst linke Hälfte sortieren,
dann rechte Hälfte sortieren,
zuletzt **sortierte** Hälften verschmelzen

merge (Verschmelzen zweier sortierter Bereiche)

merge – Beispiel

		lo			mid				hi	
help	…	1	4	6	7	2	3	5	8	…

i	j

help	…	1	4	6	7	2	3	5	8	…

help	…	1	4	6	7	2	3	5	8	…

help	…	1	4	6	7	2	3	5	8	…

help	…	1	4	6	7	2	3	5	8	…

data	…	1								…

data	…	1	2							…

data	…	1	2	3						…

data	…	1	2	3	4					…

usw.

help	…	1	4	6	7	2	3	5	8	…

data	…	1	2	3	4	5	6	7	8	…

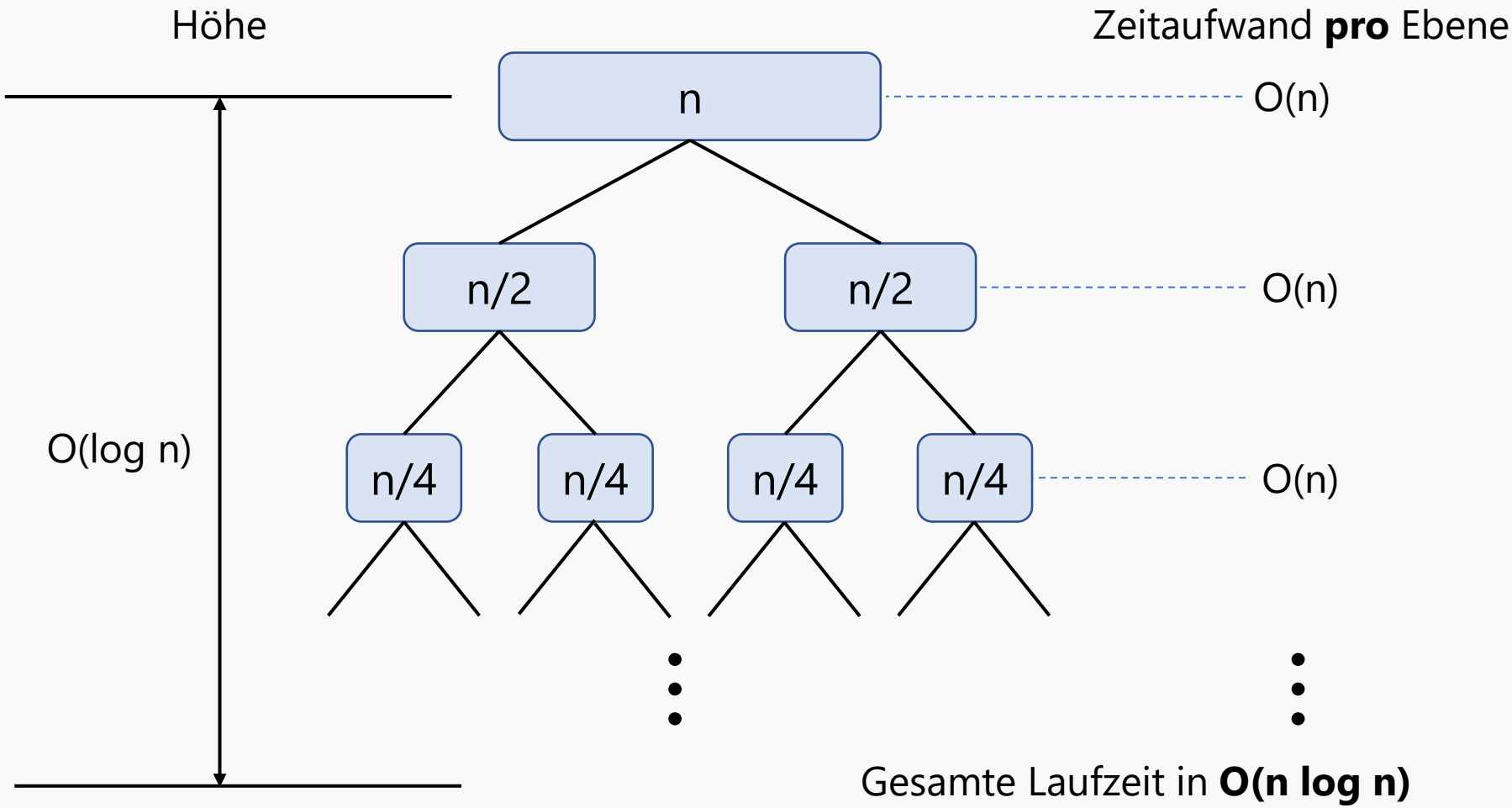
Weiteres Beispiel

lo, hi	a[] 0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	M	E	R	G	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, 0, 0, 1)	E	M	R	G	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, 2, 2, 3)	E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, 0, 1, 3)	E	G	M	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, 4, 4, 5)	E	G	M	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, 6, 6, 7)	E	G	M	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, 4, 5, 7)	E	G	M	R	E	O	R	S	T	E	X	A	M	P	L	E
merge(a, 0, 3, 7)	E	E	G	M	O	R	R	S	T	E	X	A	M	P	L	E
merge(a, 8, 8, 9)	E	E	G	M	O	R	R	S	E	T	X	A	M	P	L	E
merge(a, 10, 10, 11)	E	E	G	M	O	R	R	S	E	T	A	X	M	P	L	E
merge(a, 8, 9, 11)	E	E	G	M	O	R	R	S	A	E	T	X	M	P	L	E
merge(a, 12, 12, 13)	E	E	G	M	O	R	R	S	A	E	T	X	M	P	L	E
merge(a, 14, 14, 15)	E	E	G	M	O	R	R	S	A	E	T	X	M	P	E	L
merge(a, 12, 13, 15)	E	E	G	M	O	R	R	S	A	E	T	X	E	L	M	P
merge(a, 8, 11, 15)	E	E	G	M	O	R	R	S	A	E	E	L	M	P	T	X
merge(a, 0, 7, 15)	A	E	E	E	E	G	L	M	M	O	P	R	R	S	T	X

Trace of merge results for top-down mergesort

https://algs4.cs.princeton.edu/22mergesort/images/mergesortTD.png

Mergesort – Analyse (1)



Mergesort – Analyse (2)

- Laufzeit ist immer in O(n log n)
- Z. B. vorsortiertes Array bringt keinen Vorteil
  - Array wird immer geteilt
  - Verschmelzen wird immer durchgeführt

Mergesort – Analyse (3)

- Mehr Speicherplatzbedarf durch Hilfsarray beim Verschmelzen
  - Aufwand liegt in O(n)
- *n log n* kann nicht unterboten werden, wenn wir das **allgemeine** Sortierproblem betrachten
  - Es müssen *n* Werte sortiert werden
  - Keine zusätzliche Information über den Wertebereich
- Mergesort ist ein stabiles Sortierverfahren

Mergesort – alternative iterative Variante

```
private static void mergeSortBU(int[] data) {
    int[] help = new int[data.length];
    for (int len = 1; len < data.length; len *= 2) {
        for (int lo = 0; lo < data.length - len; lo += len + len) {
            int mid = lo + len - 1;
            int hi = Math.min(lo + len + len - 1, data.length - 1);
            merge(data, help, lo, mid, hi);
        }
    }
}
```

- Es werden nur merge-Operationen durchgeführt
  - Zuerst 1er-Blöcke verschmelzen, dann 2er-Blöcke verschmelzen, dann 4er-Blöcke verschmelzen usw.
- Diese Variante wird als Bottom-Up-Mergesort bezeichnet

Beispiel – Arraygröße ist eine Zweierpotenz

	a[i]															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	M	E	R	G	E	S	O	R	T	E	X	A	M	P	L	E
sz = 2																
merge(a, 0, 0, 1)	E	M	R	G	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, 2, 2, 3)	E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, 4, 4, 5)	E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, 6, 6, 7)	E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, 8, 8, 9)	E	M	G	R	E	S	O	R	E	T	X	A	M	P	L	E
merge(a, 10, 10, 11)	E	M	G	R	E	S	O	R	E	T	A	X	M	P	L	E
merge(a, 12, 12, 13)	E	M	G	R	E	S	O	R	E	T	A	X	M	P	L	E
merge(a, 14, 14, 15)	E	M	G	R	E	S	O	R	E	T	A	X	M	P	E	L
sz = 4																
merge(a, 0, 1, 3)	E	G	M	R	E	S	O	R	E	T	A	X	M	P	E	L
merge(a, 4, 5, 7)	E	G	M	R	E	O	R	S	E	T	A	X	M	P	E	L
merge(a, 8, 9, 11)	E	G	M	R	E	O	R	S	A	E	T	X	M	P	E	L
merge(a, 12, 13, 15)	E	G	M	R	E	O	R	S	A	E	T	X	E	L	M	P
sz = 8																
merge(a, 0, 3, 7)	E	E	G	M	O	R	R	S	A	E	T	X	E	L	M	P
merge(a, 8, 11, 15)	E	E	G	M	O	R	R	S	A	E	E	L	M	P	T	X
sz = 16																
merge(a, 0, 7, 15)	A	E	E	E	E	G	L	M	M	O	P	R	R	S	T	X

Trace of merge results for bottom-up mergesort

https://algs4.cs.princeton.edu/22mergesort/images/mergesortBU.png

Beispiel – Arraygröße ist keine Zweierpotenz

			a[ ]											
lo	m	hi	0	1	2	3	4	5	6	7	8	9	10	11
			E	A	S	Y	Q	U	E	S	T	I	O	N
0	0	1	A	E	S	Y	Q	U	E	S	T	I	O	N
2	2	3	A	E	S	Y	Q	U	E	S	T	I	O	N
4	4	5	A	E	S	Y	Q	U	E	S	T	I	O	N
6	6	7	A	E	S	Y	Q	U	E	S	T	I	O	N
8	8	9	A	E	S	Y	Q	U	E	S	I	T	O	N
10	10	11	A	E	S	Y	Q	U	E	S	I	T	N	O
0	1	3	A	E	S	Y	Q	U	E	S	I	T	N	O
4	5	7	A	E	S	Y	E	Q	S	U	I	T	N	O
8	9	11	A	E	S	Y	E	Q	S	U	I	N	O	T
0	3	7	A	E	E	Q	S	S	U	Y	I	N	O	T
0	7	11	A	E	E	I	N	O	Q	S	S	T	U	Y
			A	E	E	I	N	O	Q	S	S	T	U	Y

https://algs4.cs.princeton.edu/22mergesort/images/mergebu-3.2.2.png

Quicksort

Quicksort – Idee

- Vorgehen (Teile-und-herrsche-Verfahren)
  - Aufteilen
    - Wähle ein Element *x* (Pivotelement) aus dem Array bzw. Abschnitt im Array
    - Teile den Abschnitt im Array in zwei Abschnitte A_1 und A_2, so dass gilt
      - A_1 enthält nur Elemente kleiner gleich x
      - A_2 enthält nur Elemente größer gleich x
  - Rekursive Aufrufe
    - Rekursiver Aufruf auf A_1 erzeugt sortierte Folge B_1
    - Rekursiver Aufruf auf A_2 erzeugt sortierte Folge B_2
    - Ergebnis am Ende im Array auf einer Rekursionsebene
      - B_1, x, B_2

Quicksort – Implementierung

```
private static void quickSort(int[] data) {
    quickSort(data, 0, data.length - 1);
}

private static void quickSort(int[] data, int lo, int hi) {
    if (hi <= lo) return;
    int j = partition(data, lo, hi);
    quickSort(data, lo, j - 1);
    quickSort(data, j + 1, hi);
}

private static int partition(int[] data, int lo, int hi) {
    int k = lo, v = data[hi];
    for (int i = k; i < hi; i++) {
        if (data[i] < v) {
            exchange(data, i, k++);
        }
    }
    exchange(data, k, hi);
    return k;
}
```

Aufruf mit einem Parameter (wie bisher)

Quicksort
Zuerst partitionieren,
dann linke Hälfte sortieren,
dann rechte Hälfte sortieren

Quicksort – Beispiel für Partitionierung

6	5	7	1	3	2	4

Pivotelement für Indexbereich 0 (lo) bis 6 (hi)

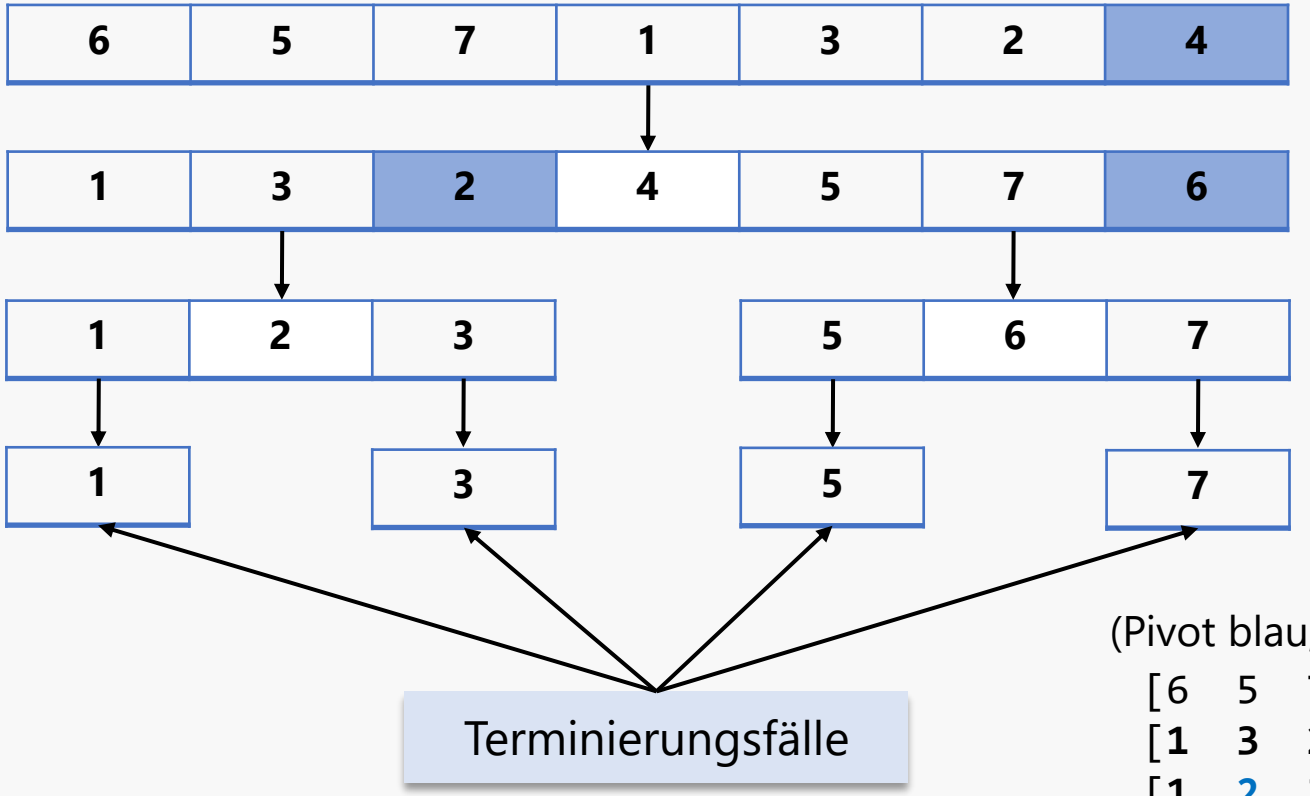
[6, 5, 7, 1, 3, 2, 4] i: 0 k: 0
[6, 5, 7, 1, 3, 2, 4] i: 1 k: 0
[6, 5, 7, 1, 3, 2, 4] i: 2 k: 0
[**1**, 5, 7, **6**, 3, 2, 4] i: 3 k: 1
[1, **3**, 7, 6, **5**, 2, 4] i: 4 k: 2
[1, 3, **2**, 6, 5, **7**, 4] i: 5 k: 3

```
private static int partition(int[] data, int lo, int hi) {
    int k = lo, v = data[hi];
    for (int i = k; i < hi; i++) {
        if (data[i] < v) {
            exchange(data, i, k++);
        }
    }
    exchange(data, k, hi);
    return k;
}
```

Letzter Aufruf von exchange tauscht Pivotelement mit dem Element an der Position k

1	3	2	4	5	7	6

Quicksort – Beispiel für optimale Aufteilung



(Pivot blau, fixierte Werte rot)

[6 5 7 1 3 2 4]
[**1** **3** **2** **4** **5** **7** **6**]
[**1** **2** **3** **4** 5 7 6]
[**1** **2** **3** **4** **5** **6** **7**]
[**1** **2** **3** **4** **5** **6** **7**]

Quicksort – schlechte Aufteilung

- Ursprüngliches Array

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

- Ausgabe jeweils nach Partitionierung (Pivot blau hervorgehoben)

[1, 2, 3, 4, 5, 6, 7, 8, 9, **10**]
[1, 2, 3, 4, 5, 6, 7, 8, **9**, 10]
[1, 2, 3, 4, 5, 6, 7, **8**, 9, 10]
[1, 2, 3, 4, 5, 6, **7**, 8, 9, 10]
[1, 2, 3, 4, 5, **6**, 7, 8, 9, 10]
[1, 2, 3, 4, **5**, 6, 7, 8, 9, 10]
[1, 2, 3, **4**, 5, 6, 7, 8, 9, 10]
[1, 2, **3**, 4, 5, 6, 7, 8, 9, 10]
[1, **2**, 3, 4, 5, 6, 7, 8, 9, 10]

Quicksort – Analyse (1)

- Schwierige Analyse
- Bester und durchschnittlicher Fall
  - **$O(n \log n)$**
- Schlechtester Fall (vorherige Folie)
  - **$O(n^2)$**
  - Durch gute/geschickte Wahl des Pivotelements kann versucht werden, den schlechtesten Fall möglichst zu vermeiden, z. B.
    - Pivotelement wird aus dem Median von drei Elementen (linkes Element, mittleres Element, rechtes Element) errechnet
    - Zufälliges Pivotelement

Quicksort – Analyse (2)

- Quicksort benötigt kein zusätzliches Hilfsarray
  - Alle Vertauschungen werden im Array (in-place) durchgeführt
- Quicksort ist nicht stabil
- Hinweis
  - Alle Aussagen beziehen sich auf die präsentierten Implementierungen!

Vergleich Sortieralgorithmen (1)

- Arrays der Länge *n* mit Zufallszahlen im Bereich 0 … *n*
- Laufzeiten für einen Algorithmus
  - Mittelwert von zehn Messungen pro Eingabegröße
  - Extrapolierte Werte (rot), abgerundete Werte (blau)
- System - Intel Core i7 9750H, Windows 10, Java 14

Vergleich Sortieralgorithmen (2)

Eingabegröße n	Bubblesort (sec)	Insertionsort (sec)	Selectionsort (sec)	Mergesort (sec)	Quicksort (sec)
10000	0,10	0,01	0,03	0,00	0,00
20000	0,43	0,05	0,09	0,00	0,00
40000	1,80	0,19	0,36	0,00	0,00
80000	7,28	0,79	1,42	0,00	0,00
160000	29,17	3,14	5,66	0,01	0,01
320000	117	13	23	0,03	0,02
640000	468	52	92	0,06	0,05
1280000	1872	208	368	0,13	0,10
2560000	7488	832	1472	0,26	0,20
5120000	29952	3328	5888	0,55	0,43
10240000	119808	13312	23552	1,13	0,89

Sortieren in linearer Zeit

Schneller sortieren

- **O(n log n)** kann unterboten werden
- Aber
  - Zusätzliche Annahme treffen bzw. Information ausnutzen
- Annahmen
  - Wir gehen von *n* Zahlen im Bereich von 0 bis *max* aus
  - $max << n$
  - Zusätzliches Hilfsarray `counts` der Länge $max + 1$

Countingsort – Idee

- Zähle in einem Hilfsarray mit, wie oft eine Zahl vorkommt
- Schreibe jede Zahl so oft in das ursprüngliche Array
  - Vereinfachte Version von Countingsort!
- Beispiel

3	**2**	**1**	**2**	**0**	**4**	**3**	**2**	**1**	**2**	**2**	**0**	**1**	**3**	**4**	**4**

Zahl/Index	**0**	**1**	**2**	**3**	**4**
Anzahl	**2**	**3**	**5**	**3**	**3**

0	**0**	**1**	**1**	**1**	**2**	**2**	**2**	**2**	**2**	**3**	**3**	**3**	**4**	**4**	**4**

Countingsort – Implementierung

```
private static void countingSort(int[] data, int max) {
    int[] counts = new int[max + 1];
    for (int i = 0; i < data.length; i++) {
        counts[data[i]]++;
    }
    int i = 0;
    for (int j = 0; j <= max; j++) {
        for (int k = 0; k < counts[j]; k++) {
            data[i++] = j;
        }
    }
}
```

O(n)

O(n)

Countingsort – Analyse

- Eine Schleife für das Befüllen des `counts`-Arrays
  - **O(n)**
- Zwei Schleifen für das Zurückschreiben
  - Es können aber nur *n* Zahlen geschrieben werden
  - **O(n)**
- Beide Teile werden hintereinander ausgeführt
  - **O(n)**
- Aber zusätzlicher Speicher notwendig
  - Vernachlässigbar, wenn $max << n$

Ausblick

Weitere Informationen

- Dieser Foliensatz bietet nur eine kurze Einführung
- Gute Startpunkte für weitere Informationen zu Algorithmen
  - https://introcs.cs.princeton.edu/java/40algorithms/
  - https://algs4.cs.princeton.edu/home/

Java

- sort-Methode in der Klasse Arrays
  - Mehrfach überladen
- Zwei Sortieralgorithmen
  - Dual-Pivot Quicksort für primitive Datentypen
  - Timsort (Variante von Mergesort) für Referenztypen